

**Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Херсонский государственный педагогический университет»**

Институт информационных технологий

Кафедра программной инженерии и информационных технологий

ПРАКТИЧЕСКАЯ РАБОТА

по дисциплине:

«Современные операционные системы и базы данных»

на тему:

«Разработка даталогической модели и структуры базы данных»

Выполнил:

Студент гр.: 12-25РПм

Трифонов Д.С.

Проверил

Преподаватель:

Алешов В.В.

Оценка: _____

Дата: 15.04.2026

ПРАКТИЧЕСКАЯ РАБОТА

Задание 1: Кафедра «Социальная работа»

ER-структура (кратко для Word)

- Сущности:
- `groups` (группы)
- `students` (студенты, связь с группой)
- `student_details` (доп. сведения, связь с `students`)
- Связи:
- группа → много студентов (1:Many)
- студент → одна запись в `student_details` (1:1)

```
MINGW64:/c/Users/admin/Desktop/test

admin@HOME-PC MINGW64 ~/Desktop/test
$ sqlite3 social_work.db
SQLite version 3.51.3 2026-03-13 10:38:09
Enter ".help" for usage hints.
sqlite>
```

```
sqlite> # Создание таблицы групп
sqlite> CREATE TABLE groups (
(x1...>   group_id   INTEGER PRIMARY KEY AUTOINCREMENT,
(x1...>   group_code TEXT NOT NULL UNIQUE
(x1...> );
sqlite>
```

```
sqlite> # Создание таблицы студентов
sqlite> CREATE TABLE students (
(x1...>   student_id   INTEGER PRIMARY KEY AUTOINCREMENT,
(x1...>   surname       TEXT NOT NULL,
(x1...>   name          TEXT NOT NULL,
(x1...>   patronymic     TEXT,
(x1...>   birth_date     TEXT NOT NULL,      -- формат 'YYYY-MM-DD'
(x1...>   funded_type    TEXT CHECK(funded_type IN ('бюджет', 'внебюджет')),

(x1...>   group_id       INTEGER,
(x1...>   FOREIGN KEY (group_id) REFERENCES groups(group_id)
(x1...> );
sqlite>
```

```
sqlite> # Создание таблицы доп. сведений
sqlite> CREATE TABLE student_details (
(x1...>   detail_id      INTEGER PRIMARY KEY AUTOINCREMENT,
(x1...>   student_id     INTEGER NOT NULL,
(x1...>   address         TEXT,
(x1...>   insurance_number TEXT,
(x1...>   passport_info   TEXT,
(x1...>   FOREIGN KEY (student_id) REFERENCES students(student_id)
(x1...> );
sqlite>
```

```

sqlite> -- Вставляем группы
sqlite> INSERT INTO groups (group_code) VALUES ('94-21');
sqlite> INSERT INTO groups (group_code) VALUES ('94-22');
sqlite> INSERT INTO groups (group_code) VALUES ('93-21');
sqlite> INSERT INTO groups (group_code) VALUES ('93-22');
sqlite> INSERT INTO groups (group_code) VALUES ('92-21');
sqlite>

```

```

sqlite> -- Вставляем студентов
sqlite> INSERT INTO students (surname, name, patronymic, birth_date, funded_type,
...> group_id)
...> VALUES ('Иванов', 'Иван', 'Иванович', '2003-05-10', 'бюджет', 1);
sqlite>
sqlite> INSERT INTO students (surname, name, patronymic, birth_date, funded_type,
...> group_id)
...> VALUES ('Петров', 'Алексей', 'Сергеевич', '2002-12-15', 'внебюджет', 1);

sqlite>
sqlite> INSERT INTO students (surname, name, patronymic, birth_date, funded_type,
...> group_id)
...> VALUES ('Сидорова', 'Мария', 'Алексеевна', '2003-03-22', 'бюджет', 2);
sqlite>

```

```

sqlite> -- Доп. сведения для студентов
sqlite> INSERT INTO student_details (student_id, address, insurance_number, pass
port_info)
...> VALUES (1, 'г. Москва, ул. Ленина, д. 1', '123-456-789', 'серия 1234 № 5
67890');
sqlite>
sqlite> INSERT INTO student_details (student_id, address, insurance_number, pass
port_info)
...> VALUES (2, 'г. Тверь, ул. Пушкина, д. 5', '987-654-321', 'серия 4321 № 0
98765');
sqlite>
sqlite> INSERT INTO student_details (student_id, address, insurance_number, pass
port_info)
...> VALUES (3, 'г. Ярославль, ул. Свободы, д. 8', '555-555-555', 'серия 5555
№ 111111');
sqlite>

```

```

sqlite> -- Вывод студентов с группой
sqlite> SELECT
...> s.student_id,
...> s.surname,
...> s.name,
...> s.funded_type,
...> g.group_code
...> FROM students s
...> JOIN groups g ON s.group_id = g.group_id;
1|Иванов|Иван|бюджет|94-21
2|Петров|Алексей|внебюджет|94-21
3|Сидорова|Мария|бюджет|94-22
sqlite>

```

```

sqlite> -- Вывод студентов с доп. сведениями
sqlite> SELECT
...> s.surname, s.name, sd.address, sd.passport_info
...> FROM students s
...> JOIN student_details sd ON s.student_id = sd.student_id;
Иванов|Иван|г. Москва, ул. Ленина, д. 1|серия 1234 № 567890
Петров|Алексей|г. Тверь, ул. Пушкина, д. 5|серия 4321 № 098765
Сидорова|Мария|г. Ярославль, ул. Свободы, д. 8|серия 5555 № 111111
sqlite>

```

Перечень контрольных вопросов

1. Какие модели называются инфологическими и реляционными?

Инфологические модели — это модели, которые описывают предметную область **на уровне понятий и отношений между объектами**, без привязки к конкретной СУБД и способу хранения данных.

Пример: модель «сущность–связь» (ER-модель), где фигурируют сущности, атрибуты и связи между объектами реального мира.

Реляционные модели — это модели, в которых данные представлены в виде **таблиц (отношений)** с rows и столбцами, а связи между таблицами реализуются через ключи.

Реляционная модель используется на уровне логического и физического проектирования баз данных (например, в MS Access, SQLite и др.).

2. В модели «сущность–связь» что понимают под сущностью и атрибутом?

Сущность — это объект или понятие реального мира, которое можно отличить от других объектов (например, «Студент», «Группа», «Пенсионер», «Предприятие»).

Набор однородных сущностей образует **множество сущностей** (таблицу в реляционной модели).

Атрибут — это свойство или характеристика сущности (например, «Фамилия», «Имя», «ИНН», «Дата рождения», «Оклад»).

В таблице атрибуту соответствует **столбец**, а конкретному значению атрибута для одной сущности — ячейка в строке.

3. Что в реляционной модели является основной структурой данных?

В реляционной модели **основной структурой данных является отношение (relation)**, которое реализуется в виде **таблицы**.

Таблица состоит из:

- **столбцов (атрибуты)** — описывают свойства сущности;
- **строк (кортежи/записи)** — хранят конкретные экземпляры сущностей.

Каждая таблица должна иметь **уникальный первичный ключ** и, при необходимости, **внешний ключ** для связи с другими таблицами.

4. Какие модели создаются после инфологического проектирования?

После построения инфологической модели создают **логическую (реляционную) модель** и **физическую модель** базы данных.

- **Реляционная (логическая) модель** — перевод ER-модели в набор таблиц, ключей и связей, уже привязанный к конкретной СУБД.
- **Физическая модель** — детальное описание того, как таблицы и индексы будут храниться на диске (тип данных, индексы, размещение файлов и т.п.).

Также иногда выделяют **внешнюю модель** — представления данных для конкретных пользователей и приложений.

5. С какой целью подключают условие «Обеспечение целостности данных» при связывании таблиц в Access?

Условие «**Обеспечение целостности данных**» в Access служит для того, чтобы **сохранить согласованность связей между таблицами**.

При его включении:

- запрещается вставлять **отсутствующие ключи** в связанные таблицы (ограничение ссылочной целостности);
- запрещается удалять или изменять записи в главной таблице, если есть связанные записи в подчинённой, если не установлены соответствующие правила каскадного удаления/изменения.

Таким образом, это условие **предотвращает появление «висячих» ссылок** и некорректных данных в базе.

6- Пройдите тест по ссылке :

<https://forms.yandex.ru/u/69df4b76eb614631bc26dbeb>

Спасибо! Ваше сообщение отправлено.